

מה יקרה אם יהיו עשרות או מאות משתמשים

התקרה האמיתית של המוצר היא סד"כ אנשי צוות של טייסת — באופן ריאלי עשרות עד כמה מאות עובדים, מנהלים בספרה בודדת, וכמה עשרות משמרות פתוחות בו-זמנית לכל חלון זמינות. בהיקף הזה:

- כל רשימה שהאפליקציה מרנדרת (עובדים, תפקידים, הסמכות, משמרות, עמוד בחינת לוח משמרות) היא לכל היותר כמה מאות שורות. Postgres מחזיר את זה במילישניות בודדות גם ללא שום כיווןן שאלות.
- ההיוריסטיקה של השיבוץ (`features/scheduling/heuristic.ts`) היא (משבצות $\times$ עובדים) 0 לכל חלון, רצה באופן סינכרוני בתוך Server Action אחת (`generateSchedule`). עבור חלון עם, למשל, 30 משמרות $\times$ 3 תפקידים כל אחת ו-100 עובדים, זה כ-9,000 בדיקות זכאות — עדיין חישוב תת-שנייתי בזיכרון, לא משהו שדורש תור או job רקע בהיקף הזה.
- ה-pooler של חיבורי Supabase (Supavisor), מופעל כברירת מחדל בכל פרויקט מתארח) כבר מטפל בבעיית חיבורים-מקבילים שפריסה serverless הייתה נתקלת בה אחרת (פונקציות Vercel פותחות חיבור DB טרי בכל הפעלה) — לא הוגדר שום דבר נוסף כי ברירת המחדל של הפלטפורמה כבר מכסה את זה בהיקף הזה. (זה נפרד מהגדרת `[db.pooler]` המקומית ב-`supabase/config.toml`, שרלוונטית רק להרצת מחסנית Docker מקומית עם `supabase start` — לא בשימוש בפרויקט הזה).

איפה זה באמת יתחיל לכאוב: לא ממספר משתמשים גולמי, אלא מפעולות מנהל שנוגעות בהרבה שורות באופן סינכרוני בבקשה אחת — המחיקה-ואז-הכנסה-מחדש של `generateSchedule` לכל שיבוצי חלון שלם, או כתיבות ההתראה-לכל-משמרת של `publishAllShiftsInWindow`. אלה גדלים עם משמרות לחלון, לא עם סך המשתמשים, ונשארים קטנים מכיוון שחלון זמינות מוגבל מבנית (תקופת שיבוץ אחת, לא כל היסטוריית הסד"כ).

אילו שאלות למסד הנתונים עלולות להיות כבדות

- (`join`) — מצרפת (`getLatestRenewalDates` (`features/worker-qualifications/queries.ts`) את `position_renews_qualifications` מול `assignments` ו-`shifts`, וכשהיא נקראת בלי `workerId` (מ-`listExpiringQualifications`, בשימוש הדשבורד של המנהל) סורקת את כל השיבוצים כדי למצוא את משמרת החידוש האחרונה לכל זוג (עובד, הסמכה). זו השאלה היחידה שכנראה תגדל באופן לינארי עם היסטוריית המשמרות הכוללת של הטייסת, מכיוון ששום דבר לא מארכב או מגביל אותה בזמן.
- `buildHeuristicInputForWindow` של `generateSchedule` — שולפת כל שורת משמרת/תפקיד/עובד/הסמכה/זמינות/אילוץ/דיווג הרלוונטית לחלון אחד בכמה שאלות, ואז מבצעת את ההתאמה בזיכרון האפליקציה במקום ב-SQL. תקין בהיקף של היום; תצטרך להיות סלקטיבית יותר (או להעביר את ההתאמה ל-SQL) אם חלון בודד אי פעם יחזיק מאות משמרות.
- widgets הדשבורד** (`features/dashboard/`) — ארבע שאלות עצמאיות (`listUnderstaffedShifts`, `listUpcomingShifts`, `listPendingApprovals`, `listExpiringQualifications`) רצות במקביל בכל טעינת דשבורד. כל אחת קטנה בפני עצמה, אבל אין caching ביניהן או בין ביקורים חוזרים — כל טעינה מריצה מחדש את כל הארבע.

האם צריך אינדקסים במסד הנתונים

כן, והם כבר קיימים, נוספו במפורש מכיוון ש-Postgres לא מאנדקס אוטומטית מפתחות זרים (רק מפתחות ראשיים/ייחודיים מקבלים אינדקס אוטומטית):

אינדקס	משרת
<code>shifts(date)</code>	כל רשימת משמרות "קרובות"/"עברו" (<code>listShifts</code>) עם <code>scope</code> , ה- <code>widget</code> של משמרות קרובות בדשבורד
<code>assignments(worker_id)</code>	<code>listMyUpcomingShifts</code> , <code>getLatestRenewalDates</code> , כל מקום ש"המשמרות של העובד הזה" נשאל
<code>worker_qualifications(worker_id)</code>	<code>listWorkerQualifications</code> , עמוד ההסמכות של העובד עצמו, עמוד הפרטים לכל עובד של המנהל
<code>notifications(worker_id, read_at)</code>	רשימת ההתראות של העובד + <code>badge</code> ספירת לא-נקרא — אינדקס מורכב כי השאילתה הזו תמיד מסננת לפי שניהם יחד
כמה אינדקסים <code>unique</code> (<code>worker_qualifications_active_unique</code> , <code>scheduling_constraints_default_unique</code> / <code>_override_unique</code> , <code>app_settings_singleton</code> וכו')	לא ממניעי סקיייל — אלה אוכפים כללים עסקיים (למשל "לכל היותר הענקת הסמכה פעילה אחת, "לכל היותר שורת אילוף ברירת מחדל אחת לסוג") ברמת ה-DB, תועלת נלווית של אינדקסים ייחודיים ולא הסיבה שהם קיימים

שום אינדקס לא נוסף באופן ספקולטיבי — כל אחד מתחקה אחרי דפוס שאילתה אמיתי שכבר קיים בקוד.

איך נמנעים מטעינה מיותרת של מידע

- כל שאילתה בוחרת עמודות בשם (`select("id, name, ...")`), לא `select("*")` — קריאות ה-`select("*")` המעטות שנותרו הן על הכנסות שורה-בודדת שאחריהן מיד `select()`. כדי לקבל בחזרה את השדות שנוצרו (`id`, `timestamps`), לא שאילתות רשימה.
- `embeds` של PostgreSQL (למשל `shifts → shift_positions → positions`) מחליפים מה שהיה אחרת N+1 סבבי בקשה בבקשה אחת — תחביר רמז-מפתח-זר מפורש (`!shift_positions_shift_id_fkey`) בשימוש בכל מקום שלטבלה יש יותר מנתיב הצטרפות אפשרי אחד לאותה יעד, עמימות אמיתית שנתקלה בה ותוקנה כמה פעמים במהלך הפיתוח.
- ה-`max_rows = 1000` של Supabase עצמה (`supabase/config.toml`) הוא תקרה קשיחה על כל תגובת PostgreSQL בודדת ללא תלות בקוד האפליקציה — כובע הגנה-בשכבות מפני שאילתה לא-חסומה בטעות, לא משהו שהאפליקציה נסמכת על להגיע אליו בפועל (כל רשימה אמיתית כאן הרבה מתחת לזה).

האם יש שימוש נכון ב-pagination

שום רשימה באפליקציה הזו לא עושה pagination היום — מגבלה ידועה ומכוונת, לא השמטה. כל רשימה (עובדים, משמרות, הסמכות, תפקידים, התראות, בחינת לוח משמרות) שולפת את כל קבוצת התוצאות ומרנדרת אותה עם מסנן חיפוש בצד לקוח מעליה, במקום שאילתה מחולקת-לעמודים בצד שרת. זו תוצאה ישירה של היקפי

הנתונים בפועל שתוארו למעלה: כמה מאות שורות המרונדרות בבת אחת לא עולות כמעט כלום לא בזמן שאילתה ולא בגודל payload, והוספת pagination עכשיו הייתה מורכבות ללא תמורה אמיתית לפני המועד האחרון. **מה היה משנה את זה:** אם סד"כ העובדים או היסטוריית המשמרות היו גדלים בסדר גודל מעבר לטייסת בודדת (רשימת ההתראות ועמוד היסטוריית המשמרות הם שני המועמדים הסבירים ביותר להצטבר ללא הגבלה לאורך זמן אמיתי, מכיוון ששניהם לא מוגבלים מבנית כמו שחלון זמינות כן), אלה יהיו שני המועמדים הראשונים ל-pagination אמיתי (מבוסס range(), ש-Supabase תומכת בו באופן טבעי).

הדוגמה הזו נהיית קונקרטית מהר יותר משנדמה: נניח, להמחשה, טייסת עם כ-50 משמרות בשבוע — `/admin/shifts/past` (שמושך את כל ההיסטוריה ללא סינון) היה חוצה כמה מאות שורות תוך כחודשיים, ואת ה-`max_rows = 1000` הקשיח של Supabase עצמה (ראו למעלה) תוך כ-20 שבוע, פחות מחצי שנה. מעבר לנקודה הזו לא רק האטה — התקרה גורמת לעמוד להציג בשקט רק 1,000 מתוך השורות האמיתיות, ללא שגיאה. כלומר תחת הנחת נפח כזו, pagination להיסטוריית משמרות היה עובר מ"שיפור עתידי כשהיה צריך" לצורך קרוב-טווח אמיתי — לא שינוי במסקנה של המסמך על היוריסטיקת השיבוץ או האינדקסים (אלה נשארים רחוקים מהיקף שמדאיג), אלא חידוד של הדחיפות בסעיף ה-pagination הספציפי הזה.

האם יש הפרדה נכונה בין צד לקוח וצד שרת

בהתאם להחלטה "ללא ספריית client-state גלובלית": Server Components שולפות ומרונדרות נתונים ישירות (אין ספריית שליפת-נתונים בצד לקוח, אין cache כפול בצד לקוח לשמור מסונכרן עם השרת); שינויים עוברים דרך Server Actions, שקוראות ל-`revalidatePath` כדי להגיד ל-Next.js אילו עמודים מרונדרים-בשרת לרענן — השרת נשאר מקור האמת היחיד. רכיבי לקוח (`"use client"`) בשימוש רק היכן שאינטראקטיביות אמיתית דורשת זאת (טפסים, ה-toggle האופטימי של זמינות, dropdowns/comboboxes) — שליפת נתונים עצמה אף פעם לא קורית בצד לקוח. זה שומר שכמות הקוד (וכך כמות הנתונים) שנשלחת לדפדפן פרופורציונלית לאינטראקטיביות, לא לנפח נתונים — רכיב הלקוח של עמוד רשימה הוא markup השורה ומטפלי האירועים, לא השורות עצמן שנשלפות מחדש בדפדפן.

אילו מגבלות קיימות במוצר הנוכחי

- אין pagination (ראו למעלה).
- אין שכבת caching בין האפליקציה ל-Postgres (אין Redis, אין cache שאילתות בזיכרון) — כל בקשה היא שאילתה חיה. תקין בהיקף הזה; יהיה הדבר הראשון להוסיף לפני pagination אם עומס קריאה אי פעם יהפוך לצוואר הבקבוק במקום נפח נתונים.
- `generateSchedule` רצה באופן סינכרוני בתוך בקשת HTTP/Server Action אחת — אין תור או worker רקע. מקובל מכיוון שכמות המשבצות בחלון בודד קטנה (ראו למעלה); בעיית שיבוץ גדולה באמת (אלפי משבצות) הייתה צריכה לעבור מ-thread הבקשה.
- ל-`getLatestRenewalDates` (ראו למעלה) אין הגבלת זמן על כמה אחורה היא מסתכלת — היא תמשיך לסרוק את כל היסטוריית השיבוצים ככל שההיסטוריה גדלה, בניגוד לכל שאילתה אחרת באפליקציה, שכולן ממוקדות באופן טבעי לנתונים "נוכחיים" או "קרובים".
- הפונקציות ה-serverless של Vercel ומסד ה-Postgres המנוהל של Supabase שניהם כבר יסודות פלטפורמה הניתנים להרחבה אופקית — שום דבר בקוד האפליקציה עצמה לא מונע היום הרחבה נוספת אם העומס אי פעם יגדל, מלבד השאילתה הספציפית למעלה.

מה הייתם משפרים בגרסה עתידית כדי לתמוך בסקייל גדול יותר

1. הגבלת זמן או pagination ל- `getLatestRenewalDates` — השאילתה היחידה בלי תקרה טבעית; הגבלתה ל-, נניח, N החודשים האחרונים של משמרות הייתה שומרת אותה שטוחה ללא תלות בכמה זמן הטייסת השתמשה באפליקציה.
2. pagination בצד שרת (`range()`) להתראות ולהיסטוריית משמרות קודם, ברגע שאחת מהן עוברת באופן ריאלי כמה מאות שורות למשתמש אחד/עמוד אחד.
3. caching לארבע שאילות ה-`widget` של הדשבורד (למשל cache בזיכרון עם TTL קצר או cache של `fetch` של Next.js) אם הדשבורד הופך לעמוד הכי מבוקר וטעינת ארבע-השאילות החוזרת שלו הופכת למדידה.
4. העברת `generateSchedule` מחוץ ל-`thread` הבקשה (job בתור, נבדק או נדחף כסטטוס) רק אם גדלי חלון בפריסה אמיתית גדלים הרבה מעבר למה שקריאה סינכרונית מטפלת בו בנוחות — לא לפני כן, מכיוון שזו מורכבות נוספת אמיתית (job runner, ממשק בדיקת סטטוס) לבעיה שהפרויקט הזה עדיין לא באמת נתקל בה.